Java Backend Architecture

Interview Series

Chapter 7.10
Build Tool CI/CD Integration

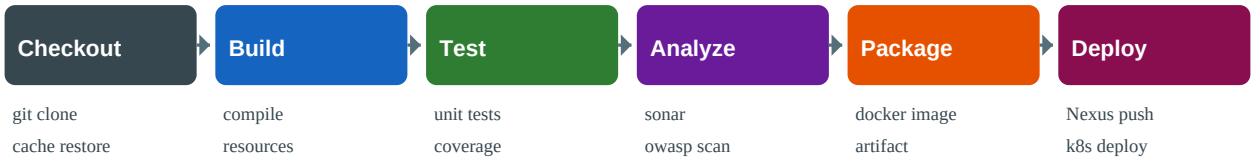
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 7.10 — Build Tool CI/CD Integration

Introduction

Integrating Maven and Gradle into CI/CD pipelines is essential for modern software delivery. Key concerns include: caching dependencies between runs, publishing artifacts to Nexus/Artifactory, containerising applications with Jib or Spring Boot Buildpacks, generating SBOMs for supply chain security, and implementing semantic versioning. This chapter covers best practices for GitHub Actions, Jenkins, and other CI systems.

CI/CD Pipeline Overview



Cache: ~/.m2 or ~/.gradle are cached between runs to avoid re-downloading dependencies

GitHub Actions — Maven

```
# .github/workflows/maven-ci.yml name: Maven CI on: push: branches: [main, develop]
pull_request: branches: [main] jobs: build: runs-on: ubuntu-latest steps: - uses:
actions/checkout@v4 - name: Set up JDK 17 uses: actions/setup-java@v4 with: java-version: "17"
distribution: "temurin" cache: maven # caches ~/.m2/repository - name: Build with Maven run:
./mvnw clean verify --batch-mode --no-transfer-progress - name: Publish Test Report uses:
dorny/test-reporter@v1 if: always() with: name: Maven Tests path: "**/surefire-reports/*.xml"
reporter: java-junit - name: Upload Coverage to Codecov uses: codecov/codecov-action@v4 with:
files: "**/jacoco.xml"
```

GitHub Actions — Gradle

```
# .github/workflows/gradle-ci.yml name: Gradle CI on: [push, pull_request] jobs: build:
runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Set up JDK 17 uses:
actions/setup-java@v4 with: java-version: "17" distribution: "temurin" - name: Setup Gradle
uses: gradle/actions/setup-gradle@v3 with: gradle-version: wrapper cache-read-only: ${{
github.ref != "refs/heads/main" }} # Only main branch writes to cache; PRs read only - name:
Build and Test run: ./gradlew build --scan - name: Upload Build Scan if: always() uses:
actions/upload-artifact@v4 with: name: build-scan-link path: build/reports/
```

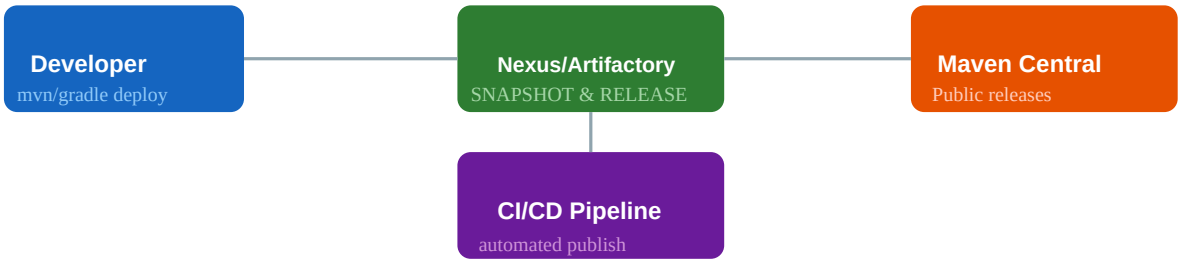
Dependency Caching Strategy

Cache Key Strategy	Maven	Gradle
Cache directory	~/.m2/repository	~/.gradle/caches, ~/.gradle/wrapper
Cache key	hashFiles('**/pom.xml')	hashFiles('**/*.gradle.kts', '**/libs.versions.toml')
Setup action	actions/setup-java cache: maven	gradle/actions/setup-gradle
Remote cache	Nexus/Artifactory proxy cache	Gradle remote build cache (HTTP)

Cache invalidation	On any pom.xml change	On build script or catalog change

```
# Manual Maven caching (alternative to setup-java cache) - name: Cache Maven packages uses: actions/cache@v4 with: path: ~/.m2 key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }} restore-keys: ${{ runner.os }}-m2 # Manual Gradle caching - name: Cache Gradle uses: actions/cache@v4 with: path: | ~/.gradle/caches ~/.gradle/wrapper key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle.kts', '**/libs.versions.toml') }} restore-keys: ${{ runner.os }}-gradle-
```

Publishing to Nexus/Artifactory




```
# Maven — settings.xml (credentials via env vars) <servers> <server> <id>nexus-snapshots</id> <username>${env.NEXUS_USER}</username> <password>${env.NEXUS_PASSWORD}</password> </server> <server> <id>nexus-releases</id> <username>${env.NEXUS_USER}</username> <password>${env.NEXUS_PASSWORD}</password> </server> </servers> <!-- pom.xml distributionManagement --> <distributionManagement> <snapshotRepository> <id>nexus-snapshots</id> <url>https://nexus.example.com/repository/maven-snapshots/</url> </snapshotRepository> <repository> <id>nexus-releases</id> <url>https://nexus.example.com/repository/maven-releases/</url> </repository> </distributionManagement> # CI deploy step ./mvnw deploy -DskipTests --batch-mode
```

```
// Gradle — build.gradle.kts publishing to Nexus plugins { `maven-publish` signing } publishing { publications { create<MavenPublication>("mavenJava") { from(components["java"]) artifact(tasks.kotlinSourcesJar) } } repositories { maven { val releasesUrl = uri("https://nexus.example.com/repository/maven-releases/") val snapshotsUrl = uri("https://nexus.example.com/repository/maven-snapshots/") url = if (version.toString().endsWith("SNAPSHOT")) snapshotsUrl else releasesUrl credentials { username = System.getenv("NEXUS_USER") password = System.getenv("NEXUS_PASSWORD") } } } } # Publish command ./gradlew publish
```

Jib Plugin in CI (Docker-free builds)

```
# GitHub Actions — build and push image with Jib (no Docker daemon needed) - name: Build and Push Docker Image env: IMAGE_TAG: ${{ github.sha }} run: | ./gradlew jib \ -Djib.to.image=gcr.io/my-project/my-app:${IMAGE_TAG} \ -Djib.to.auth.username=_json_key \ -Djib.to.auth.password="${{ secrets.GCR_JSON_KEY }}" # Spring Boot Buildpacks (also Docker-daemon-free with pack CLI) - name: Build OCI Image run: ./mvnw spring-boot:build-image \ -Dspring-boot.build-image.imageName=my-registry/my-app:${{ github.sha }}
```

SBOM Generation (CycloneDX)

```
<!-- Maven — CycloneDX SBOM --> <plugin> <groupId>org.cyclonedx</groupId> <artifactId>cyclonedx-maven-plugin</artifactId> <version>2.7.10</version> <executions> <execution> <phase>package</phase> <goals><goal>makeAggregateBom</goal></goals> </execution> </executions> <configuration> <projectType>library</projectType> <schemaVersion>1.4</schemaVersion> <includeBomSerialNumber>true</includeBomSerialNumber> <outputFormat>json</outputFormat> <outputName>bom</outputName> </configuration> </plugin> //
```

```
Gradle — CycloneDX SBOM plugins { id("org.cyclonedx.bom") version "1.8.2" } tasks.cyclonedxBom { setIncludeConfigs(listOf("runtimeClasspath")) setProjectType("application") setSchemaVersion("1.4") setDestination(project.file("build/reports")) setOutputName("bom") }
```

Semantic Versioning in CI

```
# Conventional commits → auto semantic version bump # Tools: semantic-release, release-please, axion-release (Gradle) # Gradle axion-release plugin: plugins { id("pl.allegro.tech.build.axion-release") version "1.17.0" } scmVersion { tag { prefix.set("v") versionSeparator.set("") } } version = scmVersion.version # Release commands: ./gradlew currentVersion # show current version ./gradlew release # tag and bump patch version ./gradlew release -Prelease.version=2.0.0 # explicit version # GitHub Actions semantic versioning: - name: Get version id: version run: echo "version=$(./gradlew currentVersion -q)" >> $GITHUB_OUTPUT - name: Tag release run: git tag v${{ steps.version.outputs.version }} && git push --tags
```

Jenkins Pipeline Integration

```
// Jenkinsfile (declarative pipeline) pipeline { agent any tools { jdk "JDK17" maven "Maven3.9" } environment { NEXUS_CREDS = credentials("nexus-credentials") } stages { stage("Build") { steps { sh "./mvnw clean compile --batch-mode" } } stage("Test") { steps { sh "./mvnw verify --batch-mode" } post { always { junit "**/surefire-reports/*.xml" jacoco(execPattern: "**/*.exec") } } } stage("Deploy") { when { branch "main" } steps { sh """ ./mvnw deploy --batch-mode \ -Dusername=${NEXUS_CREDS_USR} \ -Dpassword=${NEXUS_CREDS_PSW} """ } } } }
```

50+ Interview Questions & Answers

Q1. Why is caching important in CI/CD for Maven and Gradle?

Downloading all dependencies on every CI run wastes 1-5 minutes and consumes bandwidth. Caching ~/.m2 (Maven) or ~/.gradle/caches (Gradle) between runs reuses already-downloaded artifacts, making builds 2-5x faster.

Q2. What does actions/setup-java cache: maven do in GitHub Actions?

Automatically caches and restores ~/.m2/repository keyed by the hash of all pom.xml files in the repository. No manual cache action needed.

Q3. What is the gradle/actions/setup-gradle action?

GitHub Action that configures the Gradle environment, caches Gradle home, uploads build scans, and provides fine-grained cache control (cache-read-only for PRs vs write for main branch).

Q4. Why should PRs read-only from cache but not write?

PRs may use different dependency sets than main. If PRs write to the shared cache, they could corrupt the cache for main branch builds. Only main branch builds should write to the shared cache.

Q5. What is --batch-mode (Maven) and why use it in CI?

Disables interactive prompts and Maven's progress output, making logs cleaner for CI systems. Equivalent to Maven's -B flag. Always use in CI pipelines.

Q6. What is --no-transfer-progress in Maven?

Suppresses download progress output that would flood CI logs with transfer percentages. Recommended alongside --batch-mode for cleaner CI output.

Q7. What is --scan in Gradle for CI?

./gradlew build --scan uploads a detailed build scan to scans.gradle.com. Provides deep visibility into build performance, test results, and dependency resolution — valuable for debugging CI failures.

Q8. How do you publish a Maven artifact to Nexus from CI?

Configure distributionManagement in pom.xml with Nexus URL. Store credentials in CI secrets. In settings.xml: use ${env.NEXUS_USER} to read from environment. Run ./mvnw deploy -DskipTests.

Q9. How do you publish a Gradle artifact to Nexus from CI?

Apply maven-publish plugin. Configure publishing.repositories.maven with URL and credentials from environment variables. Run ./gradlew publish in CI.

Q10. What is the difference between snapshot and release repositories in Nexus?

Snapshot repo accepts versions ending in -SNAPSHOT (mutable, can be re-deployed). Release repo accepts stable versions (immutable, cannot be overwritten). Maven/Gradle select the correct repo based on the version string.

Q11. How does Jib fit into CI pipelines?

Jib builds and pushes Docker images without requiring a Docker daemon — no Docker-in-Docker (DinD) needed. Works in standard CI runners. Credentials passed via Jib properties or GOOGLE_APPLICATION_CREDENTIALS.

Q12. What is Docker-in-Docker (DinD) and why avoid it?

Running a Docker daemon inside a Docker container to build Docker images. Security risk, requires privileged containers, complex configuration. Jib and Spring Boot Buildpacks eliminate this need.

Q13. What is a SBOM and why is it important?

Software Bill of Materials — a machine-readable inventory of all components in a software artifact. Required for supply chain security (e.g., detecting Log4Shell-type vulnerabilities), compliance, and license auditing.

Q14. What format does CycloneDX SBOM use?

CycloneDX supports JSON, XML, and protobuf formats. The Maven cyclonedx-maven-plugin and Gradle org.cyclonedx.bom plugin generate CycloneDX-format SBOMs during the build.

Q15. What is semantic versioning (SemVer)?

MAJOR.MINOR.PATCH versioning: MAJOR = breaking API changes, MINOR = new backward-compatible features, PATCH = backward-compatible bug fixes. Pre-release: 1.2.3-SNAPSHOT, 1.2.3-alpha. Build metadata: 1.2.3+20240101.

Q16. What is the axion-release plugin?

A Gradle plugin that derives the project version from Git tags. Implements semantic versioning by reading the latest Git tag. Provides release tasks to tag and push new versions.

Q17. How do you automate semantic versioning with conventional commits?

Tools like semantic-release or release-please parse commit messages (feat: → MINOR, fix: → PATCH, BREAKING CHANGE: → MAJOR) and automatically determine the next version, create tags, and generate changelogs.

Q18. What is --no-transfer-progress equivalent in Gradle?

Gradle has no direct equivalent but quiet mode -q suppresses most output. The --console=plain flag provides plain output suitable for CI. configure org.gradle.console=plain in gradle.properties.

Q19. How do you handle Maven settings.xml credentials securely in CI?

Store credentials as CI secrets. Inject into settings.xml via ${env.NEXUS_USER}/${env.NEXUS_PASSWORD} — Maven reads environment variables in settings.xml. Or create settings.xml dynamically in CI pipeline from secrets.

Q20. What is the setup-java distribution parameter?

Specifies which JDK vendor to use: temurin (Eclipse Temurin/AdoptOpenJDK), microsoft, corretto (Amazon), zulu, liberica. Temurin is most common for CI builds.

Q21. How do you run only integration tests in CI?

Maven: ./mvnw verify -Dskip=true -DskipITs=false (or configure Failsafe separately). Gradle: ./gradlew integrationTest (if configured as separate task). Use CI matrix to run unit and integration tests in parallel jobs.

Q22. What is the Maven --fail-at-end (-fae) flag for CI?

Continues building all modules even if some fail, reporting all failures at the end. Useful in CI to get a complete picture of failures across all modules in a single run.

Q23. How do you publish test results in GitHub Actions?

Use dorny/test-reporter action pointing at **/surefire-reports/*.xml (Maven) or **/test-results/**/*.xml (Gradle). Creates a visible test report tab on the PR with pass/fail details.

Q24. What is Maven's --update-snapshots (-U) flag used for in CI?

Forces re-download of SNAPSHOT dependencies. Use on main branch builds to ensure the latest SNAPSHOT versions are used. Don't use on every PR build — it bypasses the cache unnecessarily.

Q25. How do you configure a Gradle remote build cache for CI?

Set up an HTTP build cache server (or Gradle Enterprise). In settings.gradle.kts: buildCache { remote { url = uri('https://cache.example.com/'); isPush = System.getenv('CI') != null } }.

Q26. What is the Gradle build cache node?

A standalone HTTP server (open-source) that implements the Gradle remote build cache protocol. Can be self-hosted for teams that don't want Gradle Enterprise.

Q27. How do you enforce that releases don't contain SNAPSHOT dependencies?

Maven: maven-enforcer-plugin with requireReleaseDeps rule (onlyWhenRelease=true). Gradle: add a custom check task that inspects configurations.runtimeClasspath.resolvedConfiguration for SNAPSHOT versions.

Q28. What is a Maven CI-friendly version in GitHub Actions?

Use the GIT_COMMIT SHA or GitHub Actions run number as the revision: ./mvnw deploy -Drevision=1.0.0 -Dchangelist=-${GITHUB_RUN_NUMBER}. Produces unique version per CI run without touching pom.xml.

Q29. How do you make Gradle builds reproducible in CI?

Enable dependency locking (./gradlew --write-locks), commit lock files, use deterministic task ordering, avoid timestamp-based inputs, use org.gradle.configureondemand=false. Enable build cache for reproducible outputs.

Q30. What is the difference between mvn package and mvn verify in CI?

package: compile + test + package (unit tests only). verify: everything in package + integration tests (Failsafe) + coverage checks. Use verify in CI for a complete quality gate before deployment.

Q31. What security scans should be part of a Java CI pipeline?

OWASP dependency-check (CVE scanning), SpotBugs/FindBugs (security rules), SonarQube (SAST), Checkstyle (code quality), secret scanning (Trufflehog, GitHub secret scanning), container scanning (Trivy)

for Docker images.